*********Running an Multiple Container using Docker Compose on AWS EC2******

Steps:-

1. Launch an EC2 Instance

  i) Log in to the AWS Management Console → Navigate to EC2 → Launch Instance.

  ii) Configure the instance as follows:

      Region: Any (example: Mumbai – ap-south-1)

      AMI (Amazon Machine Image): Amazon Linux 2023

      Instance Type: t2.micro (Free-Tier Eligible)

      Key Pair: Create or select (example: docker-compose-key)

      VPC: Default

      Availability Zone: ap-south-1a

      Security Group: Create one (example: docker-compose-sg) with rules:

      SSH (22) → Allow from your IP

      HTTP (80) → Allow from anywhere

        MYSQL/Aurora:- 3306

        Custome TCP:- 8080-9090

        All ICMP IPV4

      Storage: 20 GB is sufficient

  iii) Click Launch Instance.

2. Connect to the Instance

  Download your key pair (.pem) and convert to ppk and connect via putty

3. Install Docker and Docker Compose

  #yum install -y docker

  #systemctl enable --now docker

  #systemctl status docker

  #docker --version

Now, to install docker compose

  #mkdir -p ~/.docker/cli-plugins

  #curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \

-o ~/.docker/cli-plugins/docker-compose

  #chmod +x ~/.docker/cli-plugins/docker-compose

  #docker-compose --version

4. Create the Docker Compose File

  Create and edit a new file:

  #nano docker-compose.yml

```
services:
  nginx:
    image: nginx:stable-alpine3.21-perl
    container_name: nginx_server
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    restart: always
```

then to save {ctrl+x then press y}

image: Specifies the container image (nginx:latest).

container_name: Assigns a readable name (nginx_server).

ports: Maps host port 8080 to container port 80.

volumes: Mounts the host ./html directory to Nginx’s web root.

restart: Ensures container restarts automatically if stopped.

```
   #cat docker compose.yml
```

5. Create a Website Directory
  Check if the html directory exists:
  #ls -l /root/html
 If not, create it:
  #mkdir -p /root/html
  #echo "***Welcome to Docker-Compose Nginx Web Server***" >
/root/html/index.html

6. Start the Nginx Container
  #docker compose up -d

  Check running containers:
  #docker ps -a

  Check images:
  #docker images

7. Access the Website
  Open a browser and navigate to:
  http://<EC2-Public-IP>:8080

  You should see:

  Welcome to Docker-Compose Nginx Web Server

8. How to stop services:
   #docker compose stop

9. How to start services:

   #docker compose up -d
  (OR)
   #docker compose start

10. How to remove services:
   #docker compose down

11.How to list running/stopped containers:
   #docker ps -a

   List all images:
   #docker images

12.How to remove an image (example: Ubuntu):
   #docker rmi ubuntu

Now, To run multiple container via docker-compose.yml

```
    #nano docker-compose.yml
services:
  httpd:
    image: httpd:2.4
    container_name: httpd-container
    ports:
      - "8080:80"
    volumes:
```

```
      - ./html:/usr/local/apache2/htdocs
    restart: always

  mariadb:
    image: mariadb:latest
    container_name: mariadb-container
    environment:
      MARIADB_ROOT_PASSWORD: root123
      MARIADB_DATABASE: mydb
    ports:
      - "8081:3306"

  redis:
    image: redis:latest
    container_name: redis-container
    ports:
      - "8085:6379"


#mkdir -p /root/html
#echo "**Welcome to Web Server**" >  /root/html/index.html
#docker compose up -d     {to start the container in detached mode}
#docker compose ps         {to list the running container}
#docker comppose ps -a     {to list the runing as well as stop the container}
#docker compose stop     {to stop all container}
#docker compose stop mariadb      {To stop single container}
#docker compose start mariadb      {To start single container}

#docker compose start    {to start all container}
(OR)
docker compose up -d


#yum install -y mariadb ---------> in RHEL-9
#yum install -y mariadb105-server  ----->in amazon linux
#mysql -h 192.168.58.134 -P 8081 -u root -p
#yum install -y redis ------------->{RHEL-9}
#yum install -y redis6 -------------->{Amazon Linux}
#systemctl enable --now redis6

#redis-cli -h 192.168.58.134  -p 8085  ----------->{used this command in rhel9}
##redis6-cli -h <private ip of aws instance>  -p 8085

192.168.58.134:8085> PING
PONG
192.168.58.134:8085> KEYS *
(empty array)
192.168.58.134:8085> SET name Dinesh
OK
192.168.58.134:8085> GET NAME
(nil)
192.168.58.134:8085>
192.168.58.134:8085> GET name
"Dinesh"
192.168.58.134:8085>
192.168.58.134:8085>
192.168.58.134:8085> DBSIZE
(integer) 1
192.168.58.134:8085>
192.168.58.134:8085>
192.168.58.134:8085> exit

#docker compose down      {to remove all services}
#docker rmi image name     {to remove images}
```

Cleanup {When your practice is complete, terminate the EC2 instance from the AWS console to avoid unnecessary billing.}